



COMPUTER
ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE212/Automating Server Management	SECTION:	CPE31S2
DATE PERFORMED:	8/29/2026	DATE SUBMITTED:	8/29/2026
NAME:	Javillonar, Madhel Kathlyn Ann A.	INSTRUCTOR:	Engr. Jimlord Quejado
ACTIVITY NO. 5.1 Implement Ansible roles in playbooks			
1. PROCEDURE OUTPUT			

Task 1: Use when command for different distributions

1. In the local machine, make sure you are in the local repository directory (*CPE232_yourname*). Issue the command `git pull`. When prompted, enter the correct passphrase or password. Describe what happened when you issue this command. Did something happen? Why?

The `git pull` command was executed successfully. The output showed "Already up to date.", which means no new changes were available in the remote repository. Nothing was downloaded or updated because the local repository already contains the latest version of all files from the remote repository.

```
wdjavillonar@workstation:~$ cd ~/CPE232_Javillonar
wdjavillonar@workstation:~/CPE232_Javillonar$ pwd
/home/wdjavillonar/CPE232_Javillonar
wdjavillonar@workstation:~/CPE232_Javillonar$ git pull
Already up to date.
```

For pings

2. Edit the inventory file and add the IP address of the Centos VM. Issue the command we used to execute the playbook (the one we used in the last activity):

```
wdjavillonar@workstation:~/CPE232_Javillonar$ ansible servers -i inventory -m ping
The authenticity of host '192.168.1.23 (192.168.1.23)' can't be established.
ED25519 key fingerprint is SHA256:+a6Wj1Yfn4sLSn6KfvIR4Zu2gb6Qxqh4IDM14376pLM.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? 192.168.56.102 | SUCCE
SS => {
  "ansible_facts": {
    "discovered_interpreter_python": "/usr/bin/python3"
  },
  "changed": false,
  "ping": "pong"
}
192.168.56.101 | SUCCESS => {
  "ansible_facts": {
    "discovered_interpreter_python": "/usr/bin/python3"
  },
  "changed": false,
  "ping": "pong"
}
192.168.1.23 | UNREACHABLE! => {
  "changed": false,
  "msg": "Failed to connect to the host via ssh: Host key verification failed.",
  "unreachable": true
}
```

ansible-playbook --ask-become-pass install_apache.yml. After executing this command, you may notice that it did not become successful in the Centos VM. You can see that the Centos VM has failed=1. Only the two remote servers have been changed. The reason is that Centos VM does not support "apt" as the

package manager. The default package manager for Centos is "yum."

After adding the CentOS VM to the inventory and running the playbook, the Ubuntu servers executed the tasks successfully while the CentOS VM failed. This shows that additional configuration is required for CentOS because it uses a different package management system from Ubuntu. Ubuntu uses the apt package manager, while CentOS uses yum or dnf.

```
wdjavillonar@workstation:~/CPE232_Javillonar$ ansible-playbook -i inventory --ask-become-pass install_apache.yml
BECOME password:

PLAY [all] *****

TASK [Gathering Facts] *****
ok: [192.168.56.101]
ok: [192.168.56.102]
fatal: [192.168.1.23]: FAILED! => {"msg": "Incorrect sudo password"}

TASK [Update package cache] *****
ok: [192.168.56.101]
ok: [192.168.56.102]

TASK [Install PHP support for Apache] *****
ok: [192.168.56.102]
ok: [192.168.56.101]

PLAY RECAP *****
192.168.1.23      : ok=0    changed=0    unreachable=0    failed=1    skipped=0    rescued=0
ignored=0
192.168.56.101   : ok=3    changed=0    unreachable=0    failed=0    skipped=0    rescued=0
ignored=0
192.168.56.102   : ok=3    changed=0    unreachable=0    failed=0    skipped=0    rescued=0
ignored=0
```

3. Edit the *install_apache.yml* file and insert the lines shown below.

```
--
- hosts: all
  become: true
  tasks:

    - name: update repository index
      apt:
        update_cache: yes
        when: ansible_distribution == "Ubuntu"

    - name: install apache2 package
      apt:
        name: apache2
        when: ansible_distribution == "Ubuntu"

    - name: add PHP support for apache
      apt:
        name: libapache2-mod-php
        when: ansible_distribution == "Ubuntu"
```

After adding the when conditions to the playbook, Ansible evaluated the operating system of each host before executing the tasks. Since the tasks were configured to run only when ansible_distribution == "Ubuntu", the

CentOS host was skipped while the Ubuntu-specific tasks remained applicable only to Ubuntu systems. This prevents unsupported package manager commands from being executed on CentOS.

```
GNU nano 7.2 install_apache.yml
--
- hosts: all
  become: true

  tasks:
    - name: Update package cache
      apt:
        update_cache: yes
      when: ansible_distribution == "Ubuntu"

    - name: Install Apache
      apt:
        name: apache2
        state: latest
      when: ansible_distribution == "Ubuntu"

    - name: Install PHP support for Apache
      apt:
        name: libapache2-mod-php
        state: latest
      when: ansible_distribution == "Ubuntu"

mdjavillonar@workstation:~/CPE232_Javillonar$ ansible-playbook -i inventory --ask-become-pass ins
ache.yml
BECOME password:

PLAY [all] *****

TASK [Gathering Facts] *****
ok: [192.168.1.23]
fatal: [192.168.56.101]: FAILED! => {"msg": "Incorrect sudo password"}
fatal: [192.168.56.102]: FAILED! => {"msg": "Incorrect sudo password"}

TASK [Update package cache] *****
skipping: [192.168.1.23]

TASK [Install Apache] *****
skipping: [192.168.1.23]

TASK [Install PHP support for Apache] *****
skipping: [192.168.1.23]

PLAY RECAP *****
192.168.1.23      : ok=1    changed=0    unreachable=0    failed=0    skipped=3    rescu
ignored=0
192.168.56.101   : ok=0    changed=0    unreachable=0    failed=1    skipped=0    rescu
ignored=0
192.168.56.102   : ok=0    changed=0    unreachable=0    failed=1    skipped=0    rescu
ignored=0
```

Make sure to save the file and exit.

Run *ansible-playbook --ask-become-pass install_apache.yml* and describe the result.

If you have a mix of Debian and Ubuntu servers, you can change the configuration of your playbook like this.

```
- name: update repository index
apt:
  update_cache: yes
  when: ansible_distribution in ["Debian", "Ubuntu"]
```

Note: This will work also if you try. Notice the changes are highlighted.

4. Edit the *install_apache.yml* file and insert the lines shown below.

```
---
- hosts: all
  become: true
  tasks:

    - name: update repository index
      apt:
        update_cache: yes
        when: ansible_distribution == "Ubuntu"

    - name: install apache2 package
      apt:
        name: apache2
        state: latest
        when: ansible_distribution == "Ubuntu"

    - name: add PHP support for apache
      apt:
        name: libapache2-mod-php
        state: latest
        when: ansible_distribution == "Ubuntu"

    - name: update repository index
      dnf:
        update_cache: yes
        when: ansible_distribution == "CentOS"

    - name: install apache2 package
      dnf:
        name: httpd
        state: latest
        when: ansible_distribution == "CentOS"

    - name: add PHP support for apache
      dnf:
        name: php
        state: latest
        when: ansible_distribution == "CentOS"
```

```
wdjavillonar@workstation:~/CPE232_Javillonar$ cat install_apache.yml
---
- hosts: all
  become: true

  tasks:
    - name: update repository index
      apt:
        update_cache: yes
      when: ansible_distribution == "Ubuntu"

    - name: install apache2 package
      apt:
        name: apache2
        state: latest
      when: ansible_distribution == "Ubuntu"

    - name: add PHP support for Apache
      apt:
        name: libapache2-mod-php
        state: latest
      when: ansible_distribution == "Ubuntu"

    - name: update repository index
      dnf:
        update_cache: yes
      when: ansible_distribution == "CentOS"

    - name: install apache2 package
      dnf:
        name: httpd
        state: latest
      when: ansible_distribution == "CentOS"
```

Make sure to save and exit.

Run *ansible-playbook --ask-become-pass install_apache.yml* and describe the result.

Output for Ubuntu VMs > sysad12345



```
wdjavillonar@workstation:~/CPE232_Javillonar$ ansible-playbook -i inventory --ask-become-pass install_apache.yml
BECOME password:

PLAY [all] *****

TASK [Gathering Facts] *****
ok: [192.168.1.23]
fatal: [192.168.56.101]: FAILED! => {"msg": "Incorrect sudo password"}
fatal: [192.168.56.102]: FAILED! => {"msg": "Incorrect sudo password"}

TASK [update repository index] *****
skipping: [192.168.1.23]

TASK [install apache2 package] *****
skipping: [192.168.1.23]

TASK [add PHP support for Apache] *****
skipping: [192.168.1.23]

TASK [update repository index] *****
^C [ERROR]: User interrupted execution
wdjavillonar@workstation:~/CPE232_Javillonar$ ansible-playbook -i inventory --ask-become-pass install_apache.yml
BECOME password:

PLAY [all] *****

TASK [Gathering Facts] *****
ok: [192.168.56.101]
ok: [192.168.56.102]
fatal: [192.168.1.23]: FAILED! => {"msg": "Incorrect sudo password"}

TASK [update repository index] *****
changed: [192.168.56.102]
changed: [192.168.56.101]

TASK [install apache2 package] *****
ok: [192.168.56.102]
ok: [192.168.56.101]

TASK [add PHP support for Apache] *****
ok: [192.168.56.102]
ok: [192.168.56.101]

TASK [update repository index] *****
skipping: [192.168.56.101]
skipping: [192.168.56.102]

TASK [install apache2 package] *****
skipping: [192.168.56.101]
skipping: [192.168.56.102]

TASK [add PHP support for apache] *****
skipping: [192.168.56.101]
skipping: [192.168.56.102]

PLAY RECAP *****
192.168.1.23      : ok=0    changed=0    unreachable=0    failed=1    skipped=0    rescued=0    ignored=0
192.168.56.101   : ok=4    changed=1    unreachable=0    failed=0    skipped=3    rescued=0    ignored=0
192.168.56.102   : ok=4    changed=1    unreachable=0    failed=0    skipped=3    rescued=0    ignored=0
```

Output fo CentOS VM > sysad123

```
wdjavillonar@workstation:~/CPE232_Javillonar$ ansible-playbook -i inventory --ask-become-pass install_apache.yml
BECOME password:

PLAY [all] *****

TASK [Gathering Facts] *****
Fatal: [192.168.56.101]: FAILED! => {"msg": "Incorrect sudo password"}
Fatal: [192.168.56.102]: FAILED! => {"msg": "Incorrect sudo password"}
ok: [192.168.1.23]

TASK [update repository index] *****
skipping: [192.168.1.23]

TASK [install apache2 package] *****
skipping: [192.168.1.23]

TASK [add PHP support for Apache] *****
skipping: [192.168.1.23]

TASK [update repository index] *****
ok: [192.168.1.23]

TASK [install apache2 package] *****
changed: [192.168.1.23]

TASK [add PHP support for apache] *****
changed: [192.168.1.23]

PLAY RECAP *****
192.168.1.23      : ok=4  changed=2  unreachable=0  failed=0  skipped=3  rescued=0  ignored=0
192.168.56.101   : ok=0  changed=0  unreachable=0  failed=1  skipped=0  rescued=0  ignored=0
192.168.56.102   : ok=0  changed=0  unreachable=0  failed=1  skipped=0  rescued=0  ignored=0
```

Note: Two different outputs were obtained when running `ansible-playbook -i inventory --ask-become-pass install_apache.yml` because the Ubuntu servers and the CentOS server use different sudo passwords. Ansible only prompts for one become password, so entering the Ubuntu password allows the Ubuntu hosts to execute successfully while CentOS fails, and entering the CentOS password allows the CentOS host to execute successfully while the Ubuntu hosts fail.

Ubuntu VMs > `sysad12345`

CentOS VM > `sysad123`

5. To verify the installations, go to CentOS VM and type its IP address on the browser. Was it successful? The answer is no. It's because the `httpd` service or the Apache HTTP server in the CentOS is not yet active. Thus, you need to activate it first.

5.1 To activate, go to the CentOS VM terminal and enter the following:

`systemctl status httpd`

The result of this command tells you that the service is inactive.

5.2 Issue the following command to start the service:

`sudo systemctl start httpd`

(When prompted, enter the sudo password)

`sudo firewall-cmd --add-port=80/tcp`

(The result should be a success)

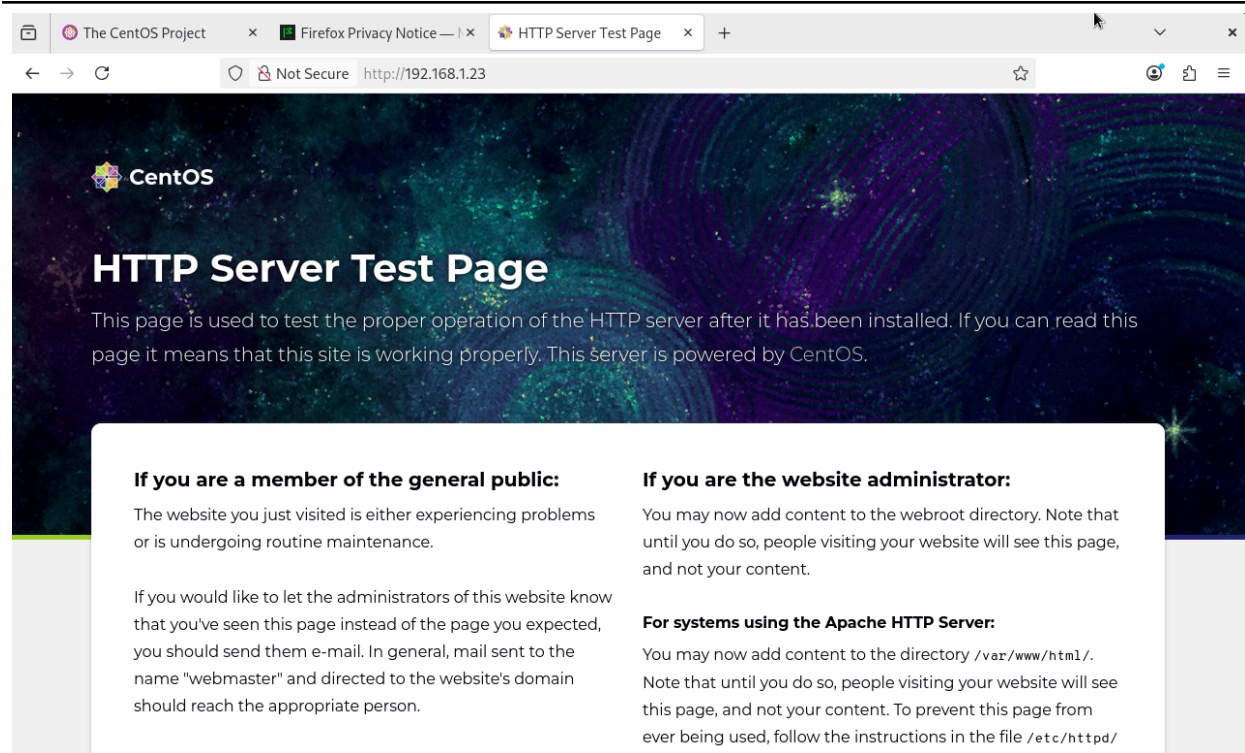
```
wdjavillonar@workstation:~/CPE232_Javillonar$ ssh cjavillonar@192.168.1.23
Activate the web console with: systemctl enable --now cockpit.socket

Last login: Sat Aug 29 15:10:11 2026 from 192.168.1.136
[cjavillonar@localhost ~]$ systemctl status httpd
○ httpd.service - The Apache HTTP Server
   Loaded: loaded (/usr/lib/systemd/system/httpd.service; disabled; preset: disabled)
   Drop-In: /usr/lib/systemd/system/httpd.service.d
            └─php-fpm.conf
   Active: inactive (dead)
   Docs: man:httpd.service(8)
[cjavillonar@localhost ~]$

[cjavillonar@localhost ~]$ sudo systemctl start httpd
[sudo] password for cjavillonar:
[cjavillonar@localhost ~]$ sudo firewall-cmd --add-port=80/tcp
success
[cjavillonar@localhost ~]$ systemctl status httpd
● httpd.service - The Apache HTTP Server
   Loaded: loaded (/usr/lib/systemd/system/httpd.service; disabled; preset: disabled)
   Drop-In: /usr/lib/systemd/system/httpd.service.d
            └─php-fpm.conf
   Active: active (running) since Sat 2026-08-29 15:19:51 PST; 43s ago
   Docs: man:httpd.service(8)
  Main PID: 36771 (httpd)
    Status: "Total requests: 0; Idle/Busy workers 100/0;Requests/sec: 0; Bytes served/sec: 0 B/sec"
     Tasks: 177 (limit: 10500)
    Memory: 18.7M (peak: 18.9M)
       CPU: 160ms
    CGroup: /system.slice/httpd.service
            └─36771 /usr/sbin/httpd -DFOREGROUND
              36778 /usr/sbin/httpd -DFOREGROUND
              36779 /usr/sbin/httpd -DFOREGROUND
              36780 /usr/sbin/httpd -DFOREGROUND
              36781 /usr/sbin/httpd -DFOREGROUND

Aug 29 15:19:51 localhost.localdomain systemd[1]: Starting The Apache HTTP Server...
Aug 29 15:19:51 localhost.localdomain httpd[36771]: AH00558: httpd: Could not reliably determine the server's fully qualified domain name, setting 'localhost'
Aug 29 15:19:51 localhost.localdomain systemd[1]: Started The Apache HTTP Server.
```

5.3 To verify the service is already running, go to CentOS VM and type its IP address on the browser. Was it successful? (Screenshot the browser)



Task 2: Refactoring playbook

This time, we want to make sure that our playbook is efficient and that the codes are easier to read. This will also makes run ansible more quickly if it has to execute fewer tasks to do the same thing.

1.Edit the playbook *install_apache.yml*. Currently, we have three tasks targeting our Ubuntu machines and 3 tasks targeting our CentOS machine. Right now, we try to consolidate some tasks that are typically the same. For example, we can consolidate two plays that install packages. We can do that by creating a list of installation packages as shown below:

```

---
- hosts: all
  become: true
  tasks:

    - name: update repository index Ubuntu
      apt:
        update_cache: yes
        when: ansible_distribution == "Ubuntu"

    - name: install apache2 and php packages for Ubuntu
      apt:
        name:
          - apache2
          - libapache2-mod-php
        state: latest
        when: ansible_distribution == "Ubuntu"

    - name: update repository index for CentOS
      dnf:
        update_cache: yes
        when: ansible_distribution == "CentOS"

    - name: install apache and php packages for CentOS
      dnf:
        name:
          - httpd
          - php
        state: latest
        when: ansible_distribution == "CentOS"

```

Make sure to save the file and exit

Run *ansible-playbook --ask-become-pass install_apache.yml* and describe the result.

2. Edit the playbook *install_apache.yml* again. In task 2.1, we consolidated the plays into one play. This time we can actually consolidated everything in just 2 plays. This can be done by removing the update repository play and putting the command *update_cache: yes* below the command *state: latest*. See below for reference:

```

---
- hosts: all
  become: true
  tasks:

    - name: install apache2 and php packages for Ubuntu
      apt:
        name:
          - apache2
          - libapache2-mod-php
        state: latest
        update_cache: yes
      when: ansible_distribution == "Ubuntu"

    - name: install apache and php packages for CentOS
      dnf:
        name:
          - httpd
          - php
        state: latest
        update_cache: yes
      when: ansible_distribution == "CentOS"

```

Make sure to save the file and exit.

Run *ansible-playbook --ask-become-pass install_apache.yml* and describe the result.

3. Finally, we can consolidate these 2 plays in just 1 play. This can be done by declaring variables that will represent the packages that we want to install. Basically, the `apache_package` and `php_package` are variables. The names are arbitrary, which means we can choose different names. We also take out the line `when: ansible_distribution`. Edit the playbook *install_apache.yml* again and make sure to follow the below image. Make sure to save the file and exit

```

---
- hosts: all
  become: true
  tasks:

    - name: install apache and php
      apt:
        name:
          - "{{ apache_package }}"
          - "{{ php_package }}"
        state: latest
        update_cache: yes

```

Run *ansible-playbook --ask-become-pass install_apache.yml* and describe the result.

4. Unfortunately, task 2.3 was not successful. It's because we need to change something in the inventory file so that the variables we declared will be in place. Edit the *inventory* file and follow the below configuration:

```
192.168.56.120 apache_package=apache2 php_package=libapache2-mod-php
192.168.56.121 apache_package=apache2 php_package=libapache2-mod-php
192.168.56.122 apache_package=httpd php_package=php
```

Make sure to save the *inventory* file and exit.

Finally, we still have one more thing to change in our *install_apache.yml* file. In task 2.3, you may notice that the package is assign as *apt*, which will not run in CentOS. Replace the *apt* with *package*. Package is a module in ansible that is generic, which is going to use whatever package manager the underlying host or the target server uses. For Ubuntu it will automatically use *apt*, and for CentOS it will automatically use *dnf*. Make sure to save the file and exit. For more details about the ansible package, you may refer to this documentation: [ansible.builtin.package – Generic OS package manager — Ansible Documentation](#)

Run *ansible-playbook --ask-become-pass install_apache.yml* and describe the result.

OUTPUTS FOR TASK 2

2.1

```
GNU nano 7.2                                install_apache.yml *
---
- hosts: all
  become: true

  tasks:
    - name: update repository index Ubuntu
      apt:
        update_cache: yes
        when: ansible_distribution == "Ubuntu"

    - name: install apache2 and php packages for Ubuntu
      apt:
        name:
          - apache2
          - libapache2-mod-php
        state: latest
        when: ansible_distribution == "Ubuntu"

    - name: update repository index for CentOS
      dnf:
        update_cache: yes
        when: ansible_distribution == "CentOS"

    - name: install apache2 and packages for CentOS
      dnf:
        name:
          - httpd
          - php
```

Ubuntu VMs > sysad12345

```
wdjavillonar@workstation:~/CPE232_Javillonar$ ansible-playbook -i inventory --ask-become-pass install_apache.yml
BECOME password:

PLAY [all] *****

TASK [Gathering Facts] *****
ok: [192.168.56.102]
ok: [192.168.56.101]
fatal: [192.168.1.23]: FAILED! => {"msg": "Incorrect sudo password"}

TASK [update repository index Ubuntu] *****
changed: [192.168.56.101]
changed: [192.168.56.102]

TASK [install apache2 and php packages for Ubuntu] *****
ok: [192.168.56.101]
ok: [192.168.56.102]

TASK [update repository index for CentOS] *****
skipping: [192.168.56.101]
skipping: [192.168.56.102]

TASK [install apache2 and packages for CentOS] *****
skipping: [192.168.56.101]
skipping: [192.168.56.102]

PLAY RECAP *****
192.168.1.23      : ok=0    changed=0    unreachable=0    failed=1    skipped=0    rescued=0    ignored=0
192.168.56.101   : ok=3    changed=1    unreachable=0    failed=0    skipped=2    rescued=0    ignored=0
192.168.56.102   : ok=3    changed=1    unreachable=0    failed=0    skipped=2    rescued=0    ignored=0
```

CentOS VM > sysad123

```
wdjavillonar@workstation:~/CPE232_Javillonar$ ansible-playbook -i inventory --ask-become-pass install_apache.yml
BECOME password:

PLAY [all] *****

TASK [Gathering Facts] *****
fatal: [192.168.56.101]: FAILED! => {"msg": "Incorrect sudo password"}
fatal: [192.168.56.102]: FAILED! => {"msg": "Incorrect sudo password"}
ok: [192.168.1.23]

TASK [update repository index Ubuntu] *****
skipping: [192.168.1.23]

TASK [install apache2 and php packages for Ubuntu] *****
skipping: [192.168.1.23]

TASK [update repository index for CentOS] *****
ok: [192.168.1.23]

TASK [install apache2 and packages for CentOS] *****
ok: [192.168.1.23]

PLAY RECAP *****
192.168.1.23      : ok=3    changed=0    unreachable=0    failed=0    skipped=2    rescued=0    ignored=0
192.168.56.101   : ok=0    changed=0    unreachable=0    failed=1    skipped=0    rescued=0    ignored=0
192.168.56.102   : ok=0    changed=0    unreachable=0    failed=1    skipped=0    rescued=0    ignored=0
```

2.2

```
--
- hosts: all
  become: true

  tasks:

    - name: install apache2 and php packages for Ubuntu
      apt:
        name:
          - apache2
          - libapache2-mod-php
        state: latest
        update_cache: yes
        when: ansible_distribution == "Ubuntu"

    - name: install apache2 and packages for CentOS
      dnf:
        name:
          - httpd
          - php
        state: latest
        update_cache: yes
        when: ansible_distribution == "CentOS"
```

The playbook was successfully refactored into two tasks by combining the package installation and repository update operations. The update cache: yes option was moved into each package installation task. Ubuntu uses the apt module while CentOS uses the dnf module, and the when conditions ensure that each task runs on the appropriate operating system.

Ubuntu VMs > sysad12345

```
wdjavillonar@workstation:~/CPE232_Javillonar$ ansible-playbook -i inventory --ask-become-pass install_apache.yml
BECOME password:

PLAY [all] *****

TASK [Gathering Facts] *****
ok: [192.168.56.102]
ok: [192.168.56.101]
fatal: [192.168.1.23]: FAILED! => {"msg": "Incorrect sudo password"}

TASK [install apache2 and php packages for Ubuntu] *****
ok: [192.168.56.101]
ok: [192.168.56.102]

TASK [install apache2 and packages for CentOS] *****
skipping: [192.168.56.101]
skipping: [192.168.56.102]

PLAY RECAP *****
192.168.1.23      : ok=0    changed=0    unreachable=0    failed=1    skipped=0    rescued=0    ignored=0
192.168.56.101   : ok=2    changed=0    unreachable=0    failed=0    skipped=1    rescued=0    ignored=0
192.168.56.102   : ok=2    changed=0    unreachable=0    failed=0    skipped=1    rescued=0    ignored=0
```

CentOS VM > sysad123

```
wdjavillonar@workstation:~/CPE232_Javillonar$ ansible-playbook -i inventory --ask-become-pass install_apache.yml
BECOME password:

PLAY [all] *****

TASK [Gathering Facts] *****
ok: [192.168.1.23]
fatal: [192.168.56.101]: FAILED! => {"msg": "Incorrect sudo password"}
fatal: [192.168.56.102]: FAILED! => {"msg": "Incorrect sudo password"}

TASK [install apache2 and php packages for Ubuntu] *****
skipping: [192.168.1.23]

TASK [install apache2 and packages for CentOS] *****
ok: [192.168.1.23]

PLAY RECAP *****
192.168.1.23      : ok=2    changed=0    unreachable=0    failed=0    skipped=1    rescued=0    ignored=0
192.168.56.101   : ok=0    changed=0    unreachable=0    failed=1    skipped=0    rescued=0    ignored=0
192.168.56.102   : ok=0    changed=0    unreachable=0    failed=1    skipped=0    rescued=0    ignored=0
```

2.3

The playbook was refactored into a single package installation task using the variables `apache_package` and `php_package`. However, the playbook failed because these variables had not yet been defined in the inventory file. Ansible therefore reported that `apache_package` was undefined.

```
GNU nano 7.2                                install_apache.yml
--
- hosts: all
  become: true

  tasks:

    - name: install apache2 and php packages for Ubuntu
      apt:
        name:
          - "[{ apache_package }]"
          - "[{ php_package }]"
        state: latest
        update_cache: yes
```

```
wdjavillonar@workstation:~/CPE232_Javillonar$ ansible-playbook -i inventory --ask-become-pass install_apache.yml
BECOME password:

PLAY [all] *****

TASK [Gathering Facts] *****
ok: [192.168.56.102]
ok: [192.168.56.101]
fatal: [192.168.1.23]: FAILED! => {"msg": "Incorrect sudo password"}

TASK [install apache2 and php packages for Ubuntu] *****
fatal: [192.168.56.101]: FAILED! => {"msg": "The task includes an option with an undefined variable. The error was: 'apache_package' is undefined. 'apache_package' is undefined\n\nThe error appears to be in '/home/wdjavillonar/CPE232_Javillonar/install_apache.yml': line 7, column 7, but may\nbe elsewhere in the file depending on the exact syntax problem.\n\nThe offending line appears to be:\n\n    - name: install apache2 and php packages for Ubuntu\n      ^ here\n"}
fatal: [192.168.56.102]: FAILED! => {"msg": "The task includes an option with an undefined variable. The error was: 'apache_package' is undefined. 'apache_package' is undefined\n\nThe error appears to be in '/home/wdjavillonar/CPE232_Javillonar/install_apache.yml': line 7, column 7, but may\nbe elsewhere in the file depending on the exact syntax problem.\n\nThe offending line appears to be:\n\n    - name: install apache2 and php packages for Ubuntu\n      ^ here\n"}

PLAY RECAP *****
192.168.1.23                : ok=0    changed=0    unreachable=0    failed=1    skipped=0    rescued=0    ignored=0
192.168.56.101             : ok=1    changed=0    unreachable=0    failed=1    skipped=0    rescued=0    ignored=0
192.168.56.102             : ok=1    changed=0    unreachable=0    failed=1    skipped=0    rescued=0    ignored=0

wdjavillonar@workstation:~/CPE232_Javillonar$ ansible-playbook -i inventory --ask-become-pass install_apache.yml
BECOME password:

PLAY [all] *****

TASK [Gathering Facts] *****
ok: [192.168.1.23]
```

2.4

I used “package” because unlike apt, the generic package module lets Ansible use the appropriate package manager automatically

```
GNU nano 7.2                                install_apache.yml *
```

```
---
- hosts: all
  become: true

  tasks:
    - name: install apache2 and php packages for Ubuntu
      package:
        name:
          - "{{ apache_package }}"
          - "{{ php_package }}"
        state: latest
        update_cache: yes
```

```
wdjavillonar@workstation:~/CPE232_Javillonar$ cat inventory
[servers]
192.168.56.101 ansible_user=serv1javillonar apache-package=apache2 php_package=libapache2-mod-php
192.168.56.102 ansible_user=serv2javillonar apache-package=apache2 php_package=libapache2-mod-php
192.168.1.23 ansible_user=cjavillonar apache-package=apache2 php_package=libapache2-mod-php
```

Ubuntu VMs > sysad12345

```
wdjavillonar@workstation:~/CPE232_javillonar$ ansible-playbook -i inventory --ask-become-pass install_apache.yml
BECOME password:

PLAY [all] *****

TASK [Gathering Facts] *****
ok: [192.168.56.101]
fatal: [192.168.1.23]: FAILED! => {"msg": "Incorrect sudo password"}
ok: [192.168.56.102]

TASK [install apache2 and php packages for Ubuntu] *****
ok: [192.168.56.102]
ok: [192.168.56.101]

PLAY RECAP *****
192.168.1.23      : ok=0    changed=0    unreachable=0    failed=1    skipped=0    rescued=0    ignored=0
192.168.56.101   : ok=2    changed=0    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
192.168.56.102   : ok=2    changed=0    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
```

CentOS VM > sysad123

```
GNU nano 7.2 install_apache.yml
---
- hosts: all
  become: true

  tasks:

    - name: install apache and php
      package:
        name:
          - "{{ apache_package }}"
          - "{{ php_package }}"
        state: latest
        update_cache: yes
```

```
GNU nano 7.2 inventory
[servers]
192.168.56.101 ansible_user=servijavillonar apache_package=apache2 php_package=libapache2-mod-php
192.168.56.102 ansible_user=serv2javillonar apache_package=apache2 php_package=libapache2-mod-php
192.168.1.23 ansible_user=cjavillonar apache_package=httpd php_package=php
```



```
wdjavillonar@workstation:~/CPE232_Javillonar$ ansible-playbook -i inventory --ask-become-pass install_apache.yml
BECOME password:

PLAY [all] *****

TASK [Gathering Facts] *****
ok: [192.168.1.23]
fatal: [192.168.56.102]: FAILED! => {"msg": "Incorrect sudo password"}
fatal: [192.168.56.101]: FAILED! => {"msg": "Incorrect sudo password"}

TASK [install apache and php] *****
ok: [192.168.1.23]

PLAY RECAP *****
192.168.1.23      : ok=2    changed=0    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
192.168.56.101   : ok=0    changed=0    unreachable=0    failed=1    skipped=0    rescued=0    ignored=0
192.168.56.102   : ok=0    changed=0    unreachable=0    failed=1    skipped=0    rescued=0    ignored=0
```

Note: Two different outputs were obtained when running `ansible-playbook -i inventory --ask-become-pass install_apache.yml` because the Ubuntu servers and the CentOS server use different sudo passwords. Ansible only prompts for one become password, so entering the Ubuntu password allows the Ubuntu hosts to execute successfully while CentOS fails, and entering the CentOS password allows the CentOS host to execute successfully while the Ubuntu hosts fail.

Ubuntu VMs > sysad12345

CentOS VM > sysad123

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Supplementary Activity:

Create a playbook that could do the previous tasks in Red Hat OS.

CentOS VM > sysad123

```
wdjavillonar@workstation:~/CPE232_Javillonar$ ansible-playbook -i inventory --ask-become-pass redhat_apache.yml
BECOME password:

PLAY [all] *****

TASK [Gathering Facts] *****
ok: [192.168.1.23]
fatal: [192.168.56.102]: FAILED! => {"msg": "Incorrect sudo password"}
fatal: [192.168.56.101]: FAILED! => {"msg": "Incorrect sudo password"}

TASK [update repository index] *****
ok: [192.168.1.23]

TASK [install Apache and PHP] *****
ok: [192.168.1.23]

PLAY RECAP *****
192.168.1.23      : ok=3    changed=0    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
192.168.56.101   : ok=0    changed=0    unreachable=0    failed=1    skipped=0    rescued=0    ignored=0
192.168.56.102   : ok=0    changed=0    unreachable=0    failed=1    skipped=0    rescued=0    ignored=0
```

```
wdjavillonar@workstation:~/CPE232_Javillonar$ cat redhat_apache.yml
---
- hosts: all
  become: true

  tasks:

    - name: update repository index
      dnf:
        update_cache: yes

    - name: install Apache and PHP
      dnf:
        name:
          - httpd
          - php
        state: latest
```

Reflections:

Answer the following:

1. Why do you think refactoring of playbook codes is important?
 - Refactoring playbook codes is important because it makes the playbook simpler, easier to read, and more efficient. It reduces repeated tasks and makes the playbook easier to maintain and modify. It can also reduce the number of tasks that Ansible needs to execute.
2. When do we use the “when” command in playbook?
 - We use the “when” command when we want a task to run only if a specific condition is true. It is useful when managing different operating systems or configurations because it allows Ansible to execute the appropriate task for a specific host.

Example from activity;

```
--
- hosts: all
  become: true

  tasks:

    - name: install apache2 and php packages for Ubuntu
      apt:
        name:
          - apache2
          - libapache2-mod-php
        state: latest
        update_cache: yes
        when: ansible_distribution == "Ubuntu"

    - name: install apache2 and packages for CentOS
      dnf:
        name:
          - httpd
          - php
        state: latest
        update_cache: yes
        when: ansible_distribution == "CentOS"
```

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: ChatGPT, QuillBot

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: ChatGPT used to ask for assistance with Ansible commands, troubleshoot and fix errors encountered during the activity, and clarify procedures. QuillBot used to improve grammar and sentence structure.

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: AI was used only for guide and I confirm it did not replace my original thinking and understanding.

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or



COMPUTER
ENGINEERING

QUEZON CITY

completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.